Лабораторная работа №6

Архитектура ЭВМ

Альманасра Рами

Content

Цель работы **5**

Задание **6**

Теоретическое введение **7**

выполнение лабораторной работы **9**

Символьные и численные данные в NASM 9
Выполнение арифметических операций в NASM 13
Answers to security questions 16
Задание для самостоятельной работы 17

Выводы **21**

Список литературы **22**

Список иллюстраций

1 перехожу в каталог 9
2 Программа вывода значения регистра eax 10
3 Запуск программы 10
4 изменение программы 11
5 Запуск программы 11
6 вторая программа 12
7 Запуск программы 12
8 вывод измененной программы 13
9 Замена функции вывода во второй программе 13
10 третья программа 14
11 Запуск третьей программы 14
12 Замена третьей программы 15
13 Запуск программы 15
14 Программа для расчета варианта 16
15 Запуск программы для расчета варианта 16
16 программа для фукции 18
17 Запуск программы 18

Список таблиц

Цель работы

Освоение арифметических инструкций языка ассемблера NASM.

Задание

1. Символьные и числовые данные в NASM
2. Выполнение арифметических операций в NASM
3. Выполнение заданий для самостоятельной работы

Теоретическое введение

Большинство инструкций на языке ассемблера требуют обработки операндов. Адрес операнда указывает местоположение, в котором хранятся обрабатываемые данные. Это могут быть данные, хранящиеся в регистре или в ячейке памяти. - Регистровая адресация - операнды хранятся в регистрах, и имена этих регистров используются в команде, например: mov ax, bx. - Прямая адресация - значение операнда указывается непосредственно в команде, например: mov ax, 2. - Адресация в памяти - операнд задает адрес в памяти. Команда задает символьное обозначение ячейки памяти, с содержимым которой необходимо выполнить операцию. Информация вводится с клавиатуры и отображается на экране в символьном виде. Эта информация кодируется в соответствии с таблицей кодов символов ASCII. ASCII - это сокращение от американского стандартного кода информации Обмен. Согласно стандарту ASCII, каждый символ кодируется одним байтом. В NASM нет инструкции, которая выводила бы числа (не в символьной форме). Поэтому, например, чтобы вывести число, вы должны сначала преобразовать его цифры в ASCII-коды этих цифр и вывести эти коды на экран, а не само число. Если вы выведете число непосредственно на экран, экран воспримет его не как число, а как последовательность символов ASCII - каждый байт числа будет восприниматься как один символ ASCII - и выведет эти символы на экран. Аналогичная ситуация возникает при вводе данных с клавиатуры. Введенные данные будут представлять собой символы, что сделает невозможным получение корректного результата при выполнении над ними арифметических действий. Для решения этой проблемы необходимо

преобразовать символы ASCII в цифры и наоборот.

выполнение лабораторной работы

Символьные и численные данные в NASM

Создаю каталог для программам лабораторной работы № 6, перехожу в него и создаю файл lab6-1.asm: (рис. -@fig:001).

Рис. 1: перехожу в каталог

В созданном файле я ввожу программу из листинга (рис. -@fig:002).

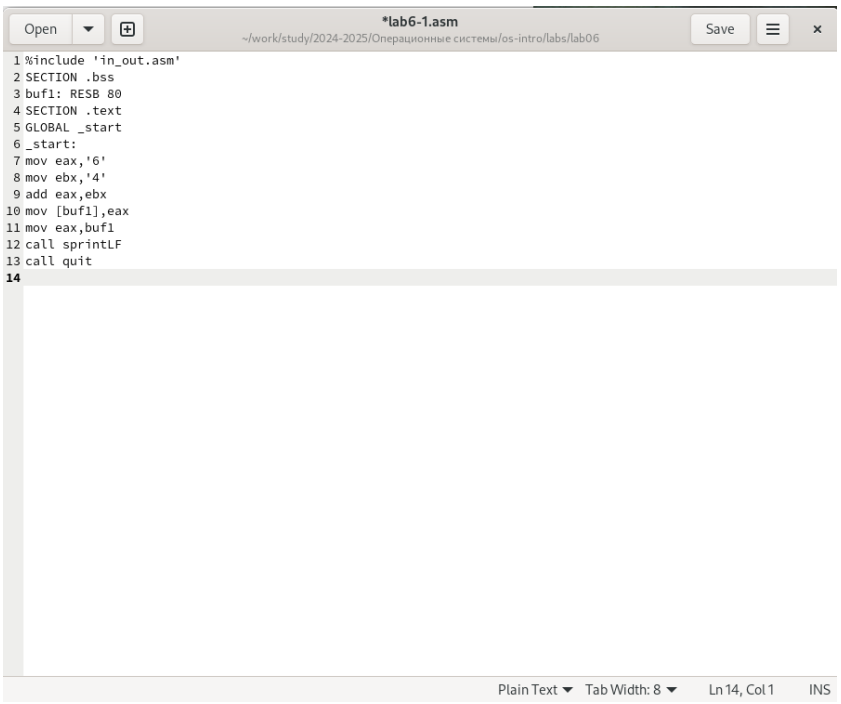

Рис. 2: Программа вывода значения регистра eax

Я создаю исполняемый файл и запускаю его, вывод программы отличается от того, что ожидалось изначально, потому что коды символов вместе дают символ j в соответствии с таблицей ASCII. {#fig:003 width=70%}

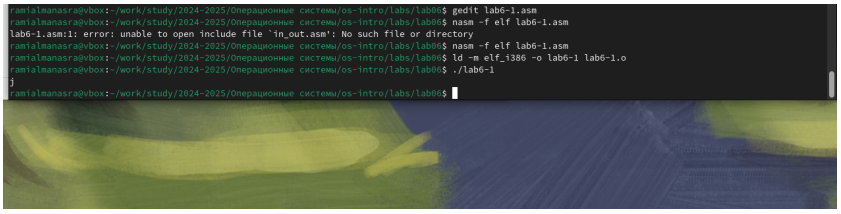

Рис. 3: Запуск программы

Я изменяю текст исходной программы, убирая кавычки (рис. -@fig:004).

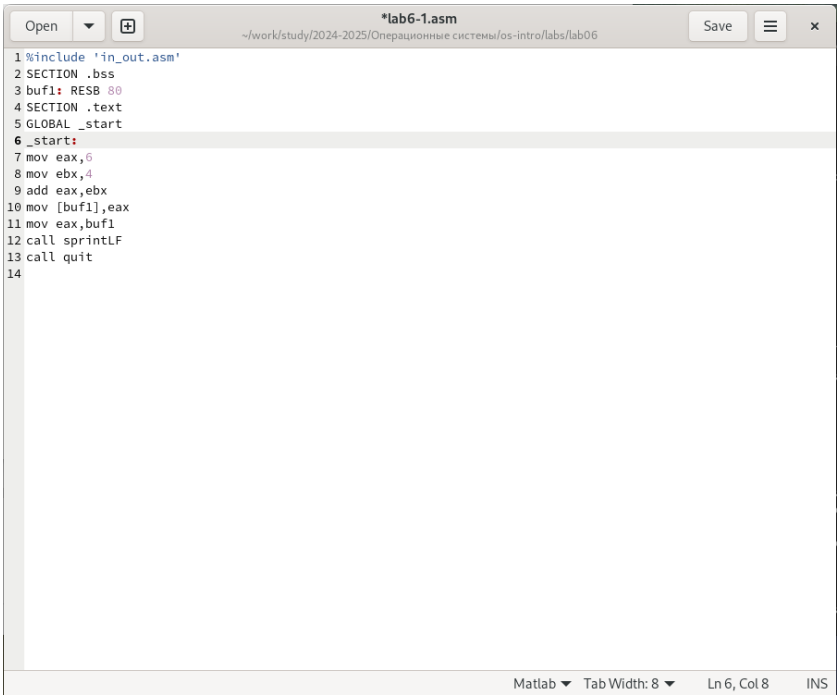

Рис. 4: изменение программы

В этот раз программа вернула пустую строку, это связано с тем, что символ 10 означает новую строку. (рис. -@fig:005).

Рис. 5: Запуск программы

Создаю файл lab6-2.asm в каталоге и ввожу в него текст программы из листинга (рис. -@fig:006).

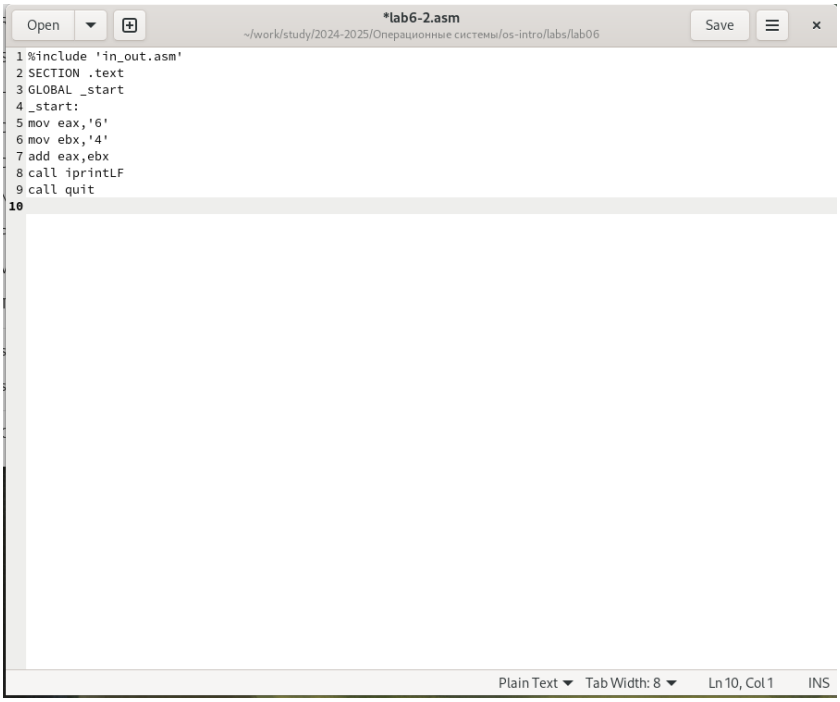

Рис. 6: вторая программа

Я создаю исполняемый файл и запускаю его, В результате работы программы мы получим число 106. В данном случае, как и в первом, команда add складывает коды символов ‘6’ и ‘4’ (54+52=106). Однако, в отличии от программы из первого листинга, функция iprintLF позволяет вывести число, а не символ, кодом которого является это число (рис. -@fig:007).

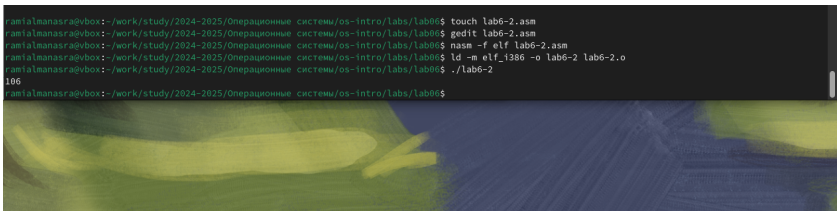

Рис. 7: Запуск программы

После удаления кавычек в программе я запускаю ее снова и получаю результат, который изначально предполагал. (рис. -@fig:008).

Рис. 8: вывод измененной программы

Замена функции вывода на iprint дает мне тот же результат, но в той же строке (рис. -@fig:009).

Рис. 9: Замена функции вывода во второй программе

Выполнение арифметических операций в NASM

Создайте файл lab6-3.asm в каталоге и ввожу текст программы из листинга(рис. -@fig:010).

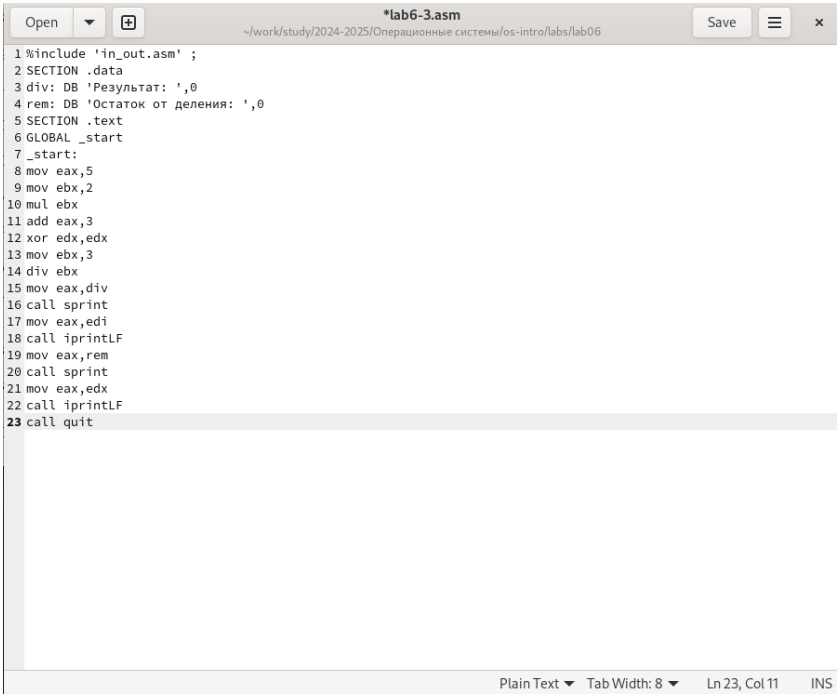

Рис. 10: третья программа

Программа выполняет арифметические вычисления, выводит результирующее выражение и его остаток от деления (рис. -@fig:011).

Рис. 11: Запуск третьей программы

Замена переменных в программе для выражения f(x) = (4*6+2)/5 (рис. -@fig:012).

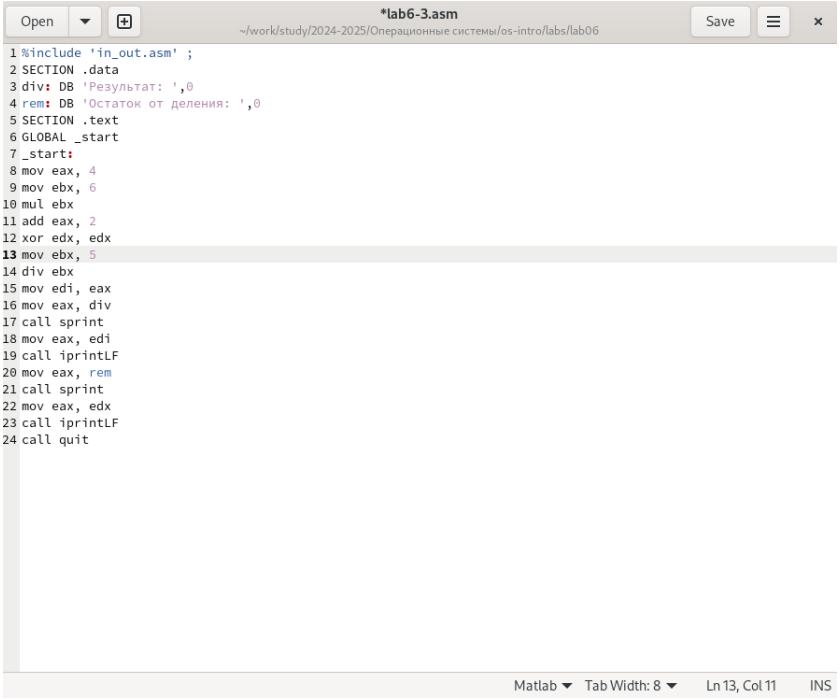

Рис. 12: Замена третьей программы

Запуск программы дает правильный результат (рис. -@fig:013).

Рис. 13: Запуск программы

Я создаю новый файл lab6-4 и помещаю текст из листинга (рис. -@fig:014).

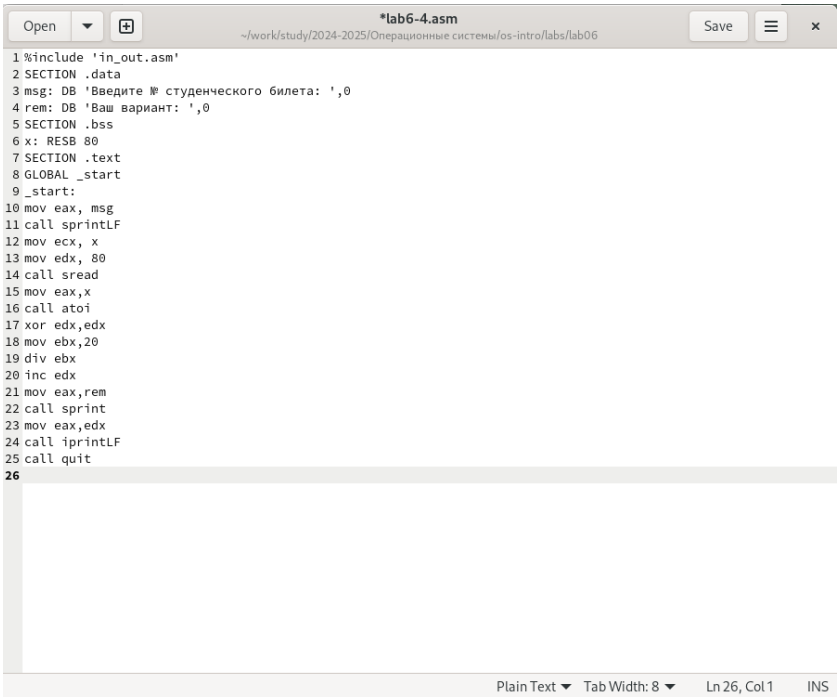

Рис. 14: Программа для расчета варианта

После запуска программы и ввода номера моего студенческого билета я получил свою версию для работы. (рис. -@fig:015).

Рис. 15: Запуск программы для расчета варианта

Answers to security questions

1. Следующие строки кода отвечают за отображение сообщения “Ваш вариант”:

```
mov eax,rem
call sprint
```

2. Команда mov ecx, x используется для ввода адреса входной строки x в регистр ecx. mov edx, 80 - записывает длину входной строки в регистр edx. call sread - вызывает подпрограмму из внешнего файла, которая обеспечивает ввод сообщения с клавиатуры.

3. call atoi используется для вызова подпрограммы из внешнего файла, которая преобразует ascii-код символа в целое число и записывает результат в регистр eax.

4. Строками, ответственными за расчет варианта, являются:

```
xor edx,edx ; reset edx for correct work div
mov ebx,20 ; ebx = 20
div ebx ; eax = eax/20, edx - remainder from division
inc edx ; edx = edx + 1
```

5.Когда выполняется команда div ebx, оставшаяся часть от деления записывается в регистр edx.

6. Команда inc edx увеличивает значение регистра edx на 1.

7. Следующие строки отвечают за отображение результатов вычислений на экране:

```
mov eax,edx
call iprintLF
```

Задание для самостоятельной работы

В соответствии с выбранным вариантом я реализую программу для вычисления функции f(x) = (2 + x)^2 , проверка по нескольким переменным показывает корректность выполнения программы (рис. -@fig:016), (рис. -@fig:017).

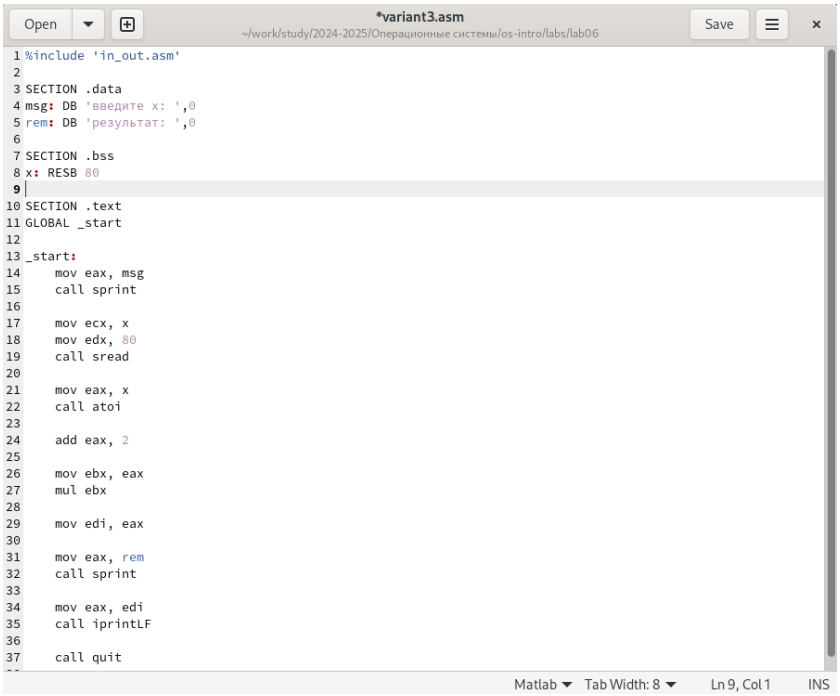

Рис. 16: программа для фукции

Рис. 17: Запуск программы

Код программы:

```
%include 'in_out.asm'

SECTION .data
msg: DB 'введите x: ',0
rem: DB 'результат: ',0
```

```
SECTION .bss
x: RESB 80

SECTION .text
GLOBAL _start

_start:
    mov eax, msg
    call sprint

    mov ecx, x
    mov edx, 80
    call sread

    mov eax, x
    call atoi

    add eax, 2

    mov ebx, eax
    mul ebx

    mov edi, eax

    mov eax, rem
    call sprint

    mov eax, edi
    call iprintLF
```

```
call quit
```

Выводы

Во время этой лабораторной работы я освоил арифметические инструкции языка ассемблера NASM.

Список литературы

1. Пример выполнения лабораторной работы
2. Курс на ТУИС
3. Лабораторная работа №6
4. Программирование на языке ассемблера NASM Столяров А. В.